

Problema 3 — Framework y SPA (1.5)

Un **componente** es una unidad independiente y reutilizable de una interfaz de usuario que encapsula su propia estructura (lo que se muestra), su lógica (cómo se comporta) y, opcionalmente, su propio estilo. La idea central es la composición: una aplicación compleja se construye combinando componentes pequeños y bien delimitados, en lugar de escribir una sola página monolítica. Un componente puede recibir datos desde afuera (comúnmente llamados "propiedades" o "props") y, a partir de ellos, decide qué renderizar. En pseudocódigo, un componente de tipo "tarjeta de usuario" podría verse así:

```
componente TarjetaUsuario(propiedades):
    nombre = propiedades.nombre
    correo = propiedades.correo
    retornar estructura:
        contenedor:
            texto(nombre)
            texto(correo)
```

Este componente se puede reutilizar tantas veces como se necesite, simplemente cambiando los datos que recibe.

El **estado** es la información interna de un componente (o de la aplicación en general) que puede cambiar con el tiempo y que, cuando cambia, provoca que la interfaz se vuelva a dibujar (re-renderizar) para reflejar ese cambio. A diferencia de las propiedades, que vienen desde afuera, el estado es propio del componente y normalmente se modifica en respuesta a eventos como clics, entradas de teclado o respuestas de una API. Un ejemplo simple sería un contador:

```
componente Contador():
    estado contador = 0

    funcion incrementar():
        contador = contador + 1
        // el framework detecta el cambio y vuelve a dibujar el componente

    retornar estructura:
        boton(al_hacer_clic = incrementar):
            texto("Valor: " + contador)
```

Cada vez que contador cambia, la interfaz se actualiza automáticamente sin que el desarrollador tenga que manipular manualmente el DOM.

El **enrutamiento** (routing) es el mecanismo que permite simular la navegación entre distintas "páginas" o vistas dentro de una aplicación, asociando una URL (o una ruta lógica) con un componente específico que debe mostrarse. En una SPA (Single Page Application), no hay recarga completa del navegador al cambiar de vista: el enrutador simplemente intercepta el cambio de URL y decide, en el lado del cliente, qué componente renderizar. En pseudocódigo:

```
rutas = {  
  "/inicio": ComponentePrincipal,  
  "/perfil": ComponentePerfil,  
  "/contacto": ComponenteContacto  
}  
  
funcion enrutador(url_actual):  
  componente = rutas[url_actual]  
  renderizar(componente)
```

Cuando el usuario hace clic en un enlace interno, el enrutador cambia la URL visible en el navegador y renderiza el componente correspondiente, todo sin solicitar una página nueva completa al servidor.

¿Por qué una SPA necesita una API?

Una SPA carga inicialmente un único archivo HTML (junto con el JavaScript y CSS asociados) y, a partir de ahí, toda la interacción ocurre en el navegador mediante manipulación del DOM y actualizaciones de estado, sin recargar la página. Sin embargo, la aplicación normalmente necesita trabajar con datos reales y persistentes (usuarios, productos, mensajes, etc.), y esos datos no pueden estar incrustados de forma estática en el HTML inicial porque cambian constantemente y deben ser compartidos entre distintos usuarios y dispositivos. Por eso, la SPA se comunica con el servidor a través de una **API** (típicamente REST o similar), enviando solicitudes asíncronas (por ejemplo, mediante fetch o AJAX) para obtener, crear, actualizar o eliminar información, sin necesidad de recargar la página completa. En pseudocódigo:

```
al_cargar_componente(ListaProductos):  
  respuesta = API.get("/productos")  
  estado productos = respuesta.datos  
  renderizar(productos)
```

En resumen, el framework se encarga de la presentación y la interactividad en el cliente, mientras que la API es el puente que conecta esa interfaz con los datos y la lógica de negocio que residen en el servidor.

English requirement (20%)

A Single Page Application (SPA) loads a single HTML file at the beginning and then updates the content dynamically using JavaScript, without reloading the entire page when the user navigates between views. In contrast, a Multi Page Application (MPA) requests a completely new HTML page from the server every time the user moves to a different section, which usually results in slower transitions but simpler server-side logic. Because of this, SPAs tend to feel faster and more fluid for the user, but they require additional mechanisms, such as client-side routing and communication with an API, to manage data and navigation properly.